CI/CD — Simple Notes

Page 1: What is CI/CD?

CI = Continuous Integration

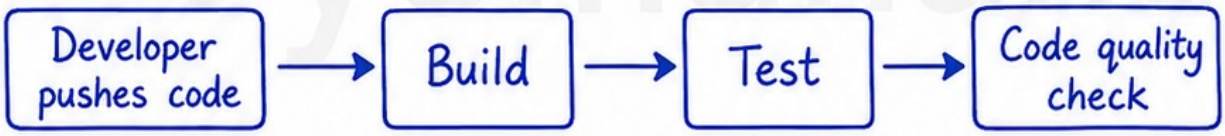
CD = Continuous Delivery / Continuous Deployment

CI means developers push code frequently and the system automatically builds and tests it.

Continuous Delivery means the application is ready to deploy after tests pass.

Continuous Deployment means the application is automatically deployed after tests pass.

CI Flow:



CD Flow:



Why CI/CD?

- Faster releases
- Less manual work
- Early bug detection
- More reliable deployments

CI/CD helps teams release software faster, safer, and more consistently.

Vyomanant Academy

CI/CD — Simple Notes

Page 2: Jenkins Complete Guide

1) Definition:

Jenkins = open-source automation server for build, test, and deploy.

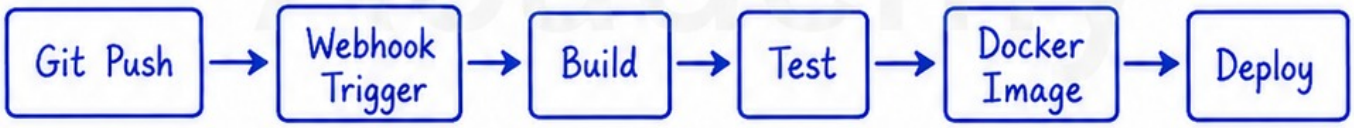
2) Architecture:



3) Main Components:

- Controller = manages jobs, pipelines, users, plugins, agents
- Agent = runs build, test, scan, deploy tasks
- Plugin = adds features like Docker, GitHub, SonarQube, Kubernetes
- Job = one automation task
- Pipeline = multiple stages together

4) Simple Jenkins Flow:



5) Why Jenkins?

- Flexible
- Large plugin ecosystem
- Pipeline as code
- Works with many tools

Jenkins automates repeated work and makes CI/CD easier.

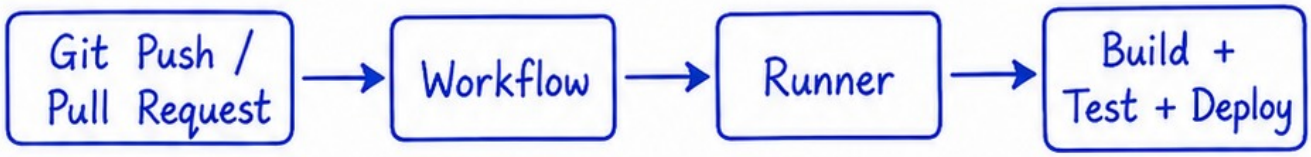
CI/CD — Simple Notes

Page 3: GitHub Actions Explained

1) Definition:

GitHub Actions = CI/CD platform built into GitHub.

2) Simple flow:



3) Important Terms:

- Workflow = automation file
- Event = push, pull request, schedule
- Job = group of steps
- Step = one task
- Runner = machine that runs workflow
- Action = reusable task

Workflow files live in: .github/workflows/

4) Simple Example:

```
name: CI
on: push
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo Build
      - run: echo Test
```

5) Why use GitHub Actions?

- built into GitHub
- easy automation
- good for pull requests
- many reusable actions

GitHub Actions connects code changes directly to automation.

CI/CD — Simple Notes

Page 4: GitLab CI Explained

1) Definition:

GitLab CI/CD automates build, test, and deploy using .gitlab-ci.yml

.gitlab-ci.yml

2) Flow:



3) Important Terms:

- Pipeline = complete workflow
- Stage = build / test / deploy
- Job = one task
- Runner = executes jobs
- Artifact = output file from a job

4) Simple Example:

```
stages:
  - build
  - test
  - deploy
build-job: echo Building
test-job: echo Testing
deploy-job: echo Deploying
```

5) Good points:

- built into GitLab
- strong pipeline stages
- supports artifacts
- good for DevOps teams

GitLab CI uses runners to execute jobs step by step.

Vamanant Academy

CI/CD — Simple Notes

Page 5: Argo CD Complete Guide

1) Definition:

Argo CD = GitOps continuous delivery tool for Kubernetes.

2) Concept: Git is the source of truth.

3) Flow in boxes:



4) Simple Idea:



5) Benefits:

- GitOps deployment
- drift detection
- rollback support
- deployment history
- web UI and CLI
- automated sync option

6) Use Argo CD for:

- Kubernetes apps
- YAML / Helm deployments
- safer cluster updates

Argo CD keeps the Kubernetes cluster aligned with Git.

Vyomanant Academy

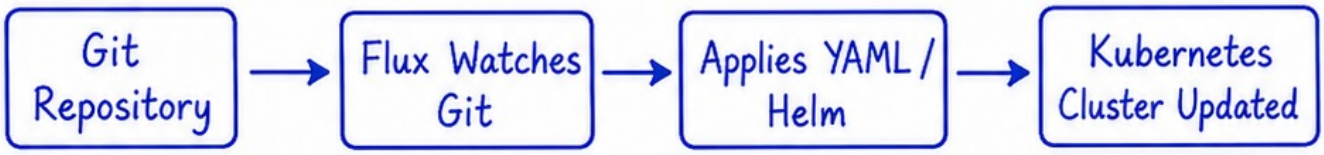
CI/CD — Simple Notes

Page 6: Flux CD Explained

1) Definition:

Flux CD = GitOps tool that watches Git and applies changes to Kubernetes.

2) Flow



3) What Flux does

- watches repositories
- applies manifests automatically
- supports Helm
- corrects drift
- works well in Kubernetes

4) Flux vs Argo CD

Argo CD	Flux CD
strong UI	lightweight approach
GitOps	GitOps
Kubernetes support	Kubernetes support
Helm support	Helm support

5) When to use Flux?

- Kubernetes-native GitOps
- teams wanting lightweight setup
- automated cluster sync

Flux CD = lightweight GitOps automation for Kubernetes.

Vyomanant Academy

CI/CD — Simple Notes

Page 7: Jenkins Pipeline

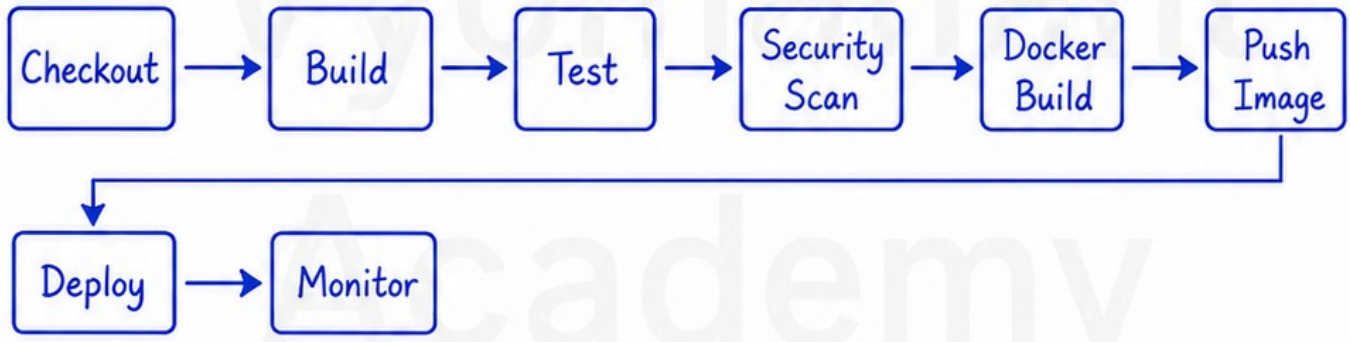
1) Definition

Pipeline = CI/CD automation written as code, usually in a Jenkinsfile.

2) Jenkinsfile

```
pipeline {
    agent any
    stages {
        stage('Build')
        stage('Test')
        stage('Deploy')
    }
}
```

3) Common Stages



4) Useful Commands

- checkout scm
- sh 'npm install'
- sh 'npm test'
- sh 'docker build -t myapp:v1 .'
- sh 'docker push registry/myapp:v1'

5) Why use pipeline as code?

- version controlled
- reusable
- easy to review
- consistent automation

Jenkins Pipeline = repeatable stages from code to deployment.

Vyomanant Academy

CI/CD — Simple Notes

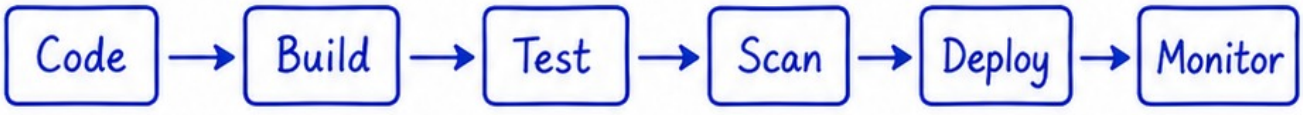
Page 8: CI/CD Best Practices

Do

- Keep pipeline code in Git
- Run tests on every push
- Use separate dev / test / stage / prod
- Scan code and Docker images
- Store secrets in secret managers
- Use versioned image tags
- Add approval before production deploy
- Keep rollback plan
- Monitor deployments
- Keep pipelines fast and simple

Avoid

- Hardcoding passwords in YAML or Jenkinsfile
- Skipping tests
- Using one environment for everything
- Using root credentials in pipeline
- Deploying from untrusted pull requests
- No rollback plan



Good CI/CD is secure, tested, repeatable, and easy to maintain.

Vyomanant Academy

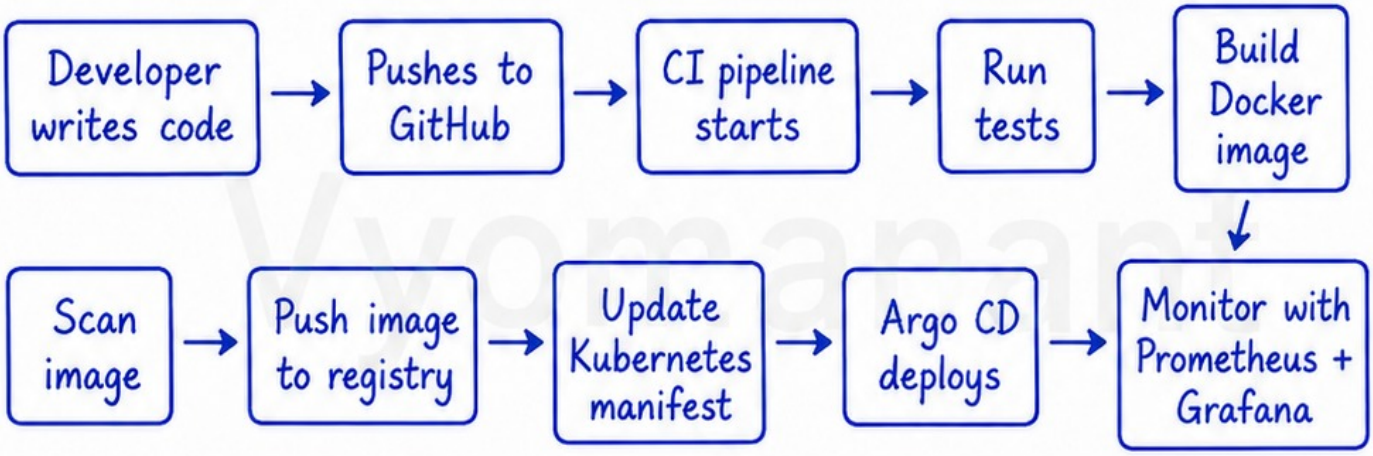
CI/CD — Simple Notes

Page 9: End-to-End CI/CD Project

Project example (tools used)

Node.js App + Docker + GitHub + GitHub Actions / Jenkins + Registry + Argo CD + Kubernetes

End-to-End CI/CD Flow



Simple Example

- Old image: myapp:v1
- New image: myapp:v2
- Update image tag in Git → Argo CD deploys v2

Why this project is useful

- shows full CI/CD flow
- combines CI and GitOps CD
- easy real-world example

End-to-end CI/CD means code moves from commit to Kubernetes automatically.

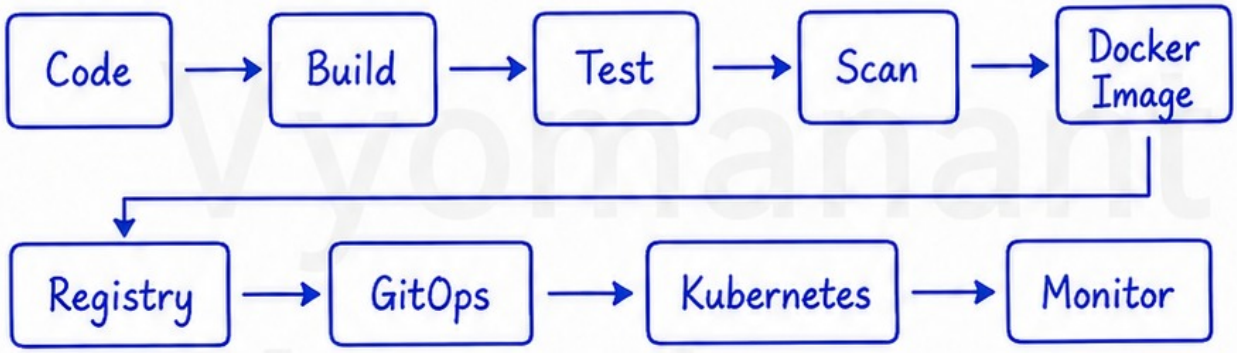
CI/CD — Simple Notes

Page 10: Quick Revision Summary

1) Tool Summary

Jenkins	automation server
Jenkinsfile	pipeline as code
GitHub Actions	CI/CD inside GitHub
GitLab CI	CI/CD inside GitLab
Argo CD	GitOps delivery for Kubernetes
Flux CD	lightweight GitOps for Kubernetes

2) CI/CD Full Flow



3) Key Idea

- CI = integrate and test code often
- CD = deliver or deploy continuously
- automation reduces manual work
- testing improves quality

4) Final Takeaway

CI/CD helps teams release software faster, safer, and more consistently.

Learn the concepts first,
then practice with a small project.

Vyomanaut Academy